AI-Powered Intelligent Interview and Placement Assessment System

Complete Beginner's Guide to System Architecture, Workflow, AI Models, Development, and Implementation

Project: HireSense

Table of Contents

1. Understanding Our Project from Zero
2. Essential Vocabulary
3. How All Parts Work Together
4. Complete User Workflow
5. Deep Dive into Our AI Models
6. Every AI Feature Separately
7. Our Actual Development Process
8. Understanding Important Files
9. How to Run and Test
10. Viva and Presentation Guide
11. Limitations and Honest Explanations

1. Understanding Our Project from Zero

1.1 What is software?

Software is a set of instructions written for a computer. A computer cannot think by itself; it follows instructions. For example, when you open a calculator app and type 2 + 2, the software tells the computer to add the numbers and show the result. Programming languages such as Python and JavaScript are ways humans write these instructions in a readable form. A compiler or interpreter translates them into machine instructions.

1.2 What is our project?

Our project is called HireSense. It is software that helps students practise for job interviews and placements. A student creates an account, uploads a resume, receives a resume analysis, practises mock interviews, receives feedback, practices English, solves assessments, and applies for jobs. Mentors guide assigned students. Recruiters manage jobs and candidates. Administrators manage the platform.

1.3 What is Artificial Intelligence?

Artificial Intelligence (AI) means software that can read and produce human-like text, find patterns, and make predictions or decisions. In our project, AI helps analyse resumes, generate interview questions, evaluate answers, explain job matches, and coach English conversation. AI is not perfect; it can be wrong, slow, or repeat itself.

1.4 What is an AI model?

An AI model is a large collection of learned patterns. It is not an ordinary function with fixed if/else rules. It receives input text and produces output text based on patterns learned during training. The application wraps the model with prompts, validation, and rules.

1.5 Did we train our own AI model?

No, we did not train our own language model. We downloaded pretrained local models and connected them through Ollama and transformers. The language model is qwen3:4b. The embedding model is sentence-transformers/all-MiniLM-L6-v2. These run on the user's computer.

We did train one small traditional machine-learning model: a question-type classifier. It is a TF-IDF plus logistic regression model trained on 176 hand-written interview questions (plus 15 admin seed questions). It predicts whether a question is technical, HR, behavioral, communication, or project. It is used only as a cross-check and never changes the interview.

Important distinction: writing a prompt is not training. Downloading a model is not training. Training only means changing model weights using a dataset and a training algorithm.

1.6 Types of model use in our project

Approach	Used?	Evidence
Trained from scratch	Yes, only question-type classifier	backend/ml/train_question_classifier.py, data/interview_questions_dev.csv
Fine-tuned pretrained model	No	No fine-tuning scripts or checkpoints found
Pretrained model run locally	Yes	Ollama qwen3:4b, all-MiniLM-L6-v2
External AI API	No	No external API key configured; all AI local
Rule-based logic	Yes	Adaptive difficulty thresholds, score caps, matching weights
Mock/partial feature	Some UI features may depend on model/browser	Verified case-by-case

System architecture diagram

```
HireSense System
```

2. Essential Vocabulary

Term	Simple meaning	In our project
Frontend	Part users see	React app
Backend	Server logic	FastAPI
Database	Stores data	SQLite/PostgreSQL
API	Way to communicate	/api/* routes
Endpoint	One API address	/api/student/interviews
JSON	Text data format	Request/response
Authentication	Who are you	Login JWT cookie
Authorization	What you can do	Role checks (student/recruiter/etc)
Model	Learned AI pattern	qwen3:4b, MiniLM
Prompt	Instruction to AI	ai.py prompts
Inference	Running a model	generate question
Embedding	Text turned into numbers	matching.py
Token	Words/pieces of text	Model input
Ollama	Local AI runtime	Runs qwen3:4b locally

3. How All Parts Work Together

Frontend

The frontend is written in React and runs in the browser. It shows pages, buttons, forms, dashboards, and charts. It collects input and sends it to the backend through API calls in src/api.js.

Backend

The backend is written in FastAPI (Python). It receives requests, checks login, validates input, runs business logic, calls AI models or rules, stores data, and returns responses. Routers map URLs to functions. Services contain reusable logic.

Database

The database stores users, profiles, resumes, interviews, questions, answers, assessments, jobs, applications, mentoring, progress, and logs. SQLite is default; PostgreSQL is supported. Tables and relationships are defined in backend/app/models.py.

APIs

An API is like a menu of operations. The frontend places an order (request) to an endpoint and gets a response. Example: POST /api/student/interviews creates a mock interview.

Authentication and roles

Authentication checks identity using JWT in an httpOnly cookie. Authorization checks role. Roles: student, recruiter (admin-approved), mentor, admin. Backend guards are in backend/app/deps.py.

4. Complete User Workflow (Rahul example)

Step 1: Registration and login

Rahul opens the React app and registers as a student. Frontend calls POST /api/auth/register then login. Backend hashes the password, stores the user, and sets a JWT cookie.

Step 2: Profile and resume

Rahul uploads a PDF/DOCX resume. Backend extracts text using pypdf or python-docx (resume_parser.py), stores it, and runs AI analysis.

Step 3: Resume analysis

The local LLM reads the resume text (wrapped in prompt fencing) and returns JSON. Backend validates and removes invented skills/projects. Resume completeness score uses a fixed points table.

Step 4: Start interview

Rahul chooses an interview type, role, and interviewer. Backend creates an interview session and generates the first question using resume facts.

Step 5: Answer

Rahul answers by text or live voice. Live voice uses browser recording, Whisper transcription (backend), then the existing answer API.

Step 6: Evaluation

Backend evaluates the answer using the LLM with fixed rule caps (empty/short answers capped). It determines next difficulty (up if score>=75, down if <45).

Step 7: Report

When complete, the system shows per-question feedback and a final report with skill breakdown.

Step 8: Mentor

Assigned mentor views progress, writes feedback, and creates improvement plans.

Step 9: Recruiter

Recruiter publishes jobs and reviews candidate applications with match scores.

Step 10: Admin

Admin manages users, settings, question bank, badges, and monitoring.

5. Deep Dive into Our AI Models

5.1 Ollama local LLM

Model: qwen3:4b via Ollama. It is a 4-billion parameter general-purpose language model. It runs locally on CPU/GPU, is Apache-2.0 licensed, and is configured by OLLAMA_MODEL, OLLAMA_BASE_URL, OLLAMA_NUM_CTX, etc. Backend service: backend/app/services/local_llm.py and ai.py.

5.2 Sentence Transformers

Model: sentence-transformers/all-MiniLM-L6-v2. It turns text into numbers (embeddings) for semantic similarity. Used in job matching and duplicate-question detection. Backend service: backend/app/services/embeddings.py, matching.py, interview_engine.py.

5.3 Question-type classifier

This is the only model trained by us. It is a scikit-learn TF-IDF + logistic regression classifier trained on 176 hand-written questions. Training script: backend/ml/train_question_classifier.py. Hold-out accuracy 0.917; 5-fold CV 0.885. It is a cross-check only.

5.4 Voice models

TTS uses Kokoro-82M (local, ~330 MB) with espeak-ng phonemes. STT uses Whisper base.en (local, ~150 MB). Browser recorder captures 16 kHz WAV with silence detection. These are downloaded on first use and run locally.

5.5 Computer vision monitoring

Camera monitoring uses MediaPipe Face Landmarker running in the browser. It analyses frames locally and stores only event records. Frames/audio are never uploaded.

AI inference workflow

```
User clicks 'Start interview'
                                                                                -> f
```

6. Every AI Feature Separately

Resume analysis

Extracts skills, education, projects, experience; validates fields against resume text; calculates completeness score.

Interview question generation

Generates personalized questions from resume facts and job role; retries near-duplicates.

Answer evaluation

Scores relevance, correctness, completeness, clarity, communication with rule-based caps.

Adaptive difficulty

Up if score>=75, down if <45, with explanations.

Job matching

Weighted score of required/preferred skills, semantic similarity, and experience, plus gap cap.

English conversation

Aria/English companion with corrections and memory; spoken or text.

Voice interview

TTS reading questions, STT transcribing answers, same scoring pipeline.

Computer vision monitoring

On-device face and tab signals; severity assigned server-side; never affects scores.

Question-type classifier

Cross-checks LLM question type; never changes interview.

7. Our Actual Development Process

- Planning: defined problem, roles, and modules from PROJECT_SPEC.md and README.md.

- Fro

8. Understanding Important Files

File	Purpose
backend/app/main.py	App creation, routers, middleware, startup warmup
backend/app/config.py	Settings from environment
backend/app/models.py	All DB tables
backend/app/schemas.py	Request validation
backend/app/services/ai.py	All AI prompts and validation
backend/app/services/matching.py	Job matching
backend/app/services/rewards.py	Points/badges/streaks
backend/app/routers/interviews.py	Interview API
frontend/src/api.js	Frontend HTTP client
frontend/src/App.jsx	Routes and guards
frontend/src/voice/*	Avatar/lip-sync/state machines

9. How to Run and Test

Prerequisites

- Python 3.12+

- No

Backend

```
cd backend
```

```
pyth
```

Frontend

```
cd frontend
```

```
npm
```

Tests

```
cd backend
```

```
.\ve
```

Backend health: GET http://127.0.0.1:8000/api/health. API docs: http://127.0.0.1:8000/docs. Frontend: http://localhost:5173.

10. Viva and Presentation Guide

A. One-minute explanation

HireSense is a local-AI-based placement preparation web app. Students upload resumes, get AI analysis, practise mock interviews with adaptive difficulty and feedback, and use English practice. Mentors, recruiters, and admins each have portals. AI runs locally with Ollama, so data stays private.

B. Five-minute explanation

Problem: students lack personalised interview practice. Solution: an AI interviewer that reads the resume, asks relevant questions, scores answers, and gives feedback. Architecture: React frontend, FastAPI backend, SQLite/PostgreSQL database, local Ollama LLM, local embeddings, local voice. Implemented features include resume analysis, interviews, assessments, job matching, mentor/admin portals. Limitations include dependence on local 4B model and browser voice support.

C. Technical viva questions (selected)

Q: What is the purpose?
A: Help students prepare for placements with AI feedback.

Q: Frontend vs backend?
A: Frontend is UI in browser; backend is server logic.

Q: What is an API?
A: Endpoint receiving requests and returning responses.

Q: Which database?
A: SQLite default, PostgreSQL supported.

Q: How are questions generated?
A: From resume facts and role via local LLM.

Q: Did we train our own AI model?
A: Only a small scikit-learn classifier; LLM is pretrained.

Q: How can we prove training?
A: ml/train_question_classifier.py and dataset.

Q: How does answer evaluation work?
A: LLM scores with rule caps for empty/short answers.

Q: What happens when model fails?
A: Interview uses question bank; answers unscored.

Q: What are limitations?
A: 4B model quality/speed; no OCR; no UI automation tests.

Q: What was my contribution as team leader?
A: Assigned/proposed: architecture, AI integration, coordination.

D. Troubleshooting note

If a demo fails, explain honestly what dependency is missing (Ollama, model, microphone) and that answers can be entered by typing.

E. Limitations

- Local 4B model is slower/less accurate than cloud models.

- Vo

11. Limitations and Honest Explanations

The implementation is real and tested, but some features depend on local model quality and browser capabilities. Always distinguish implemented vs intended vs future. Our team contribution allocation is proposed because Git history shows one contributor.